2976. Minimum Cost to Convert String I

Solved

Medium | Topics | Companies | Hint

You are given two **0-indexed** strings `source` and `target`, both of length `n` and consisting of **lowercase** English letters. You are also given two **0-indexed** character arrays `original` and `changed`, and an integer array `cost`, where `cost[i]` represents the cost of changing the character `original[i]` to the character `changed[i]`.

You start with the string `source`. In one operation, you can pick a character `x` from the string and change it to the character `y` at a cost of `z` **if** there exists **any** index `j` such that `cost[j] == z`, `original[j] == x`, and `changed[j] == y`.

Return *the **minimum** cost to convert the string* `source` *to the string* `target` *using **any** number of operations. If it is impossible to convert* `source` *to* `target`, *return* `-1`.

Note that there may exist indices `i`, `j` such that `original[j] == original[i]` and `changed[j] == changed[i]`.

Example 1:

```
Input: source = "abcd", target = "acbe", original =
["a","b","c","c","e","d"], changed = ["b","c","b","e","b","e"], cost =
[2,5,5,1,2,20]
Output: 28
Explanation: To convert the string "abcd" to string "acbe":
- Change value at index 1 from 'b' to 'c' at a cost of 5.
- Change value at index 2 from 'c' to 'e' at a cost of 1.
- Change value at index 2 from 'e' to 'b' at a cost of 2.
- Change value at index 3 from 'd' to 'e' at a cost of 20.
The total cost incurred is 5 + 1 + 2 + 20 = 28.
It can be shown that this is the minimum possible cost.
```

Example 2:

```
Input: source = "aaaa", target = "bbbb", original = ["a","c"], changed =
["c","b"], cost = [1,2]
Output: 12
Explanation: To change the character 'a' to 'b' change the character 'a'
to 'c' at a cost of 1, followed by changing the character 'c' to 'b' at
a cost of 2, for a total cost of 1 + 2 = 3. To change all occurrences of
'a' to 'b', a total cost of 3 * 4 = 12 is incurred.
```

Example 3:

> **Input:** source = "abcd", target = "abce", original = ["a"], changed = ["e"], cost = [10000]
> **Output:** -1
> **Explanation:** It is impossible to convert source to target because the value at index 3 cannot be changed from 'd' to 'e'.

Constraints:

- `1 <= source.length == target.length <= 10^5`
- `source`, `target` consist of lowercase English letters.
- `1 <= cost.length == original.length == changed.length <= 2000`
- `original[i]`, `changed[i]` are lowercase English letters.
- `1 <= cost[i] <= 10^6`
- `original[i] != changed[i]`

Python:

```
class Solution:
    def minimumCost(self, source: str, target: str, original: list[str], changed: list[str], cost: list[int]) -> int:
        inf = float('inf')
        dist = [[inf] * 26 for _ in range(26)]

        for i in range(26):
            dist[i][i] = 0

        for o, c, z in zip(original, changed, cost):
            u = ord(o) - 97
            v = ord(c) - 97
            dist[u][v] = min(dist[u][v], z)

        for k in range(26):
            for i in range(26):
                if dist[i][k] == inf:
                    continue
                for j in range(26):
                    if dist[k][j] != inf:
                        dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

```
        total_cost = 0
        for s_char, t_char in zip(source, target):
            u = ord(s_char) - 97
            v = ord(t_char) - 97
            if u == v:
                continue
            if dist[u][v] == inf:
                return -1
            total_cost += dist[u][v]

        return total_cost
```

JavaScript:

```
/**
 * @param {string} source
 * @param {string} target
 * @param {character[]} original
 * @param {character[]} changed
 * @param {number[]} cost
 * @return {number}
 */
var minimumCost = function(source, target, original, changed, cost) {
    const CHAR_COUNT = 26;
    const INF = Number.MAX_SAFE_INTEGER / 2;

    const buildConversionGraph = (original, changed, cost) => {
        const graph = Array(CHAR_COUNT).fill().map(() => Array(CHAR_COUNT).fill(INF));
        for (let i = 0; i < CHAR_COUNT; i++) {
            graph[i][i] = 0;
        }
        for (let i = 0; i < cost.length; i++) {
            const from = original[i].charCodeAt(0) - 'a'.charCodeAt(0);
            const to = changed[i].charCodeAt(0) - 'a'.charCodeAt(0);
            graph[from][to] = Math.min(graph[from][to], cost[i]);
        }
        return graph;
    };

    const optimizeConversionPaths = (graph) => {
        for (let k = 0; k < CHAR_COUNT; k++) {
            for (let i = 0; i < CHAR_COUNT; i++) {
                if (graph[i][k] < INF) {
                    for (let j = 0; j < CHAR_COUNT; j++) {
```

```
                    if (graph[k][j] < INF) {
                        graph[i][j] = Math.min(graph[i][j], graph[i][k] + graph[k][j]);
                    }
                }
            }
        }
    };

    const computeTotalConversionCost = (source, target, graph) => {
        let totalCost = 0;
        for (let i = 0; i < source.length; i++) {
            const sourceChar = source.charCodeAt(i) - 'a'.charCodeAt(0);
            const targetChar = target.charCodeAt(i) - 'a'.charCodeAt(0);
            if (sourceChar !== targetChar) {
                if (graph[sourceChar][targetChar] === INF) {
                    return -1;
                }
                totalCost += graph[sourceChar][targetChar];
            }
        }
        return totalCost;
    };

    const conversionGraph = buildConversionGraph(original, changed, cost);
    optimizeConversionPaths(conversionGraph);
    return computeTotalConversionCost(source, target, conversionGraph);
};
```

Java:

```
import java.util.Arrays;

class Solution {
    public long minimumCost(String source, String target,
char[] original, char[] changed, int[] cost) {
        long[][] dist = new long[26][26];
        long INF = Long.MAX_VALUE / 2;

        for (long[] row : dist) {
            Arrays.fill(row, INF);
        }
        for (int i = 0; i < 26; i++) {
            dist[i][i] = 0;
        }
```

```
        for (int i = 0; i < original.length; i++) {
            int u = original[i] - 'a';
            int v = changed[i] - 'a';
            dist[u][v] = Math.min(dist[u][v], cost[i]);
        }

        for (int k = 0; k < 26; k++) {
            for (int i = 0; i < 26; i++) {
                if (dist[i][k] == INF) continue;
                for (int j = 0; j < 26; j++) {
                    if (dist[k][j] != INF) {
                        dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
                    }
                }
            }
        }

        long totalCost = 0;
        int n = source.length();

        for (int i = 0; i < n; i++) {
            int u = source.charAt(i) - 'a';
            int v = target.charAt(i) - 'a';
            if (u == v) continue;
            if (dist[u][v] == INF) return -1;
            totalCost += dist[u][v];
        }

        return totalCost;
    }
}
```